

Bazele Inteligenței Artificiale

Lab 10. Rețele neuronale recurente

Efrem Dragoș-Sebastian-Mihaly

Procesarea limbajului natural

- Știm cu toții de ChatGPT, Gemini sau Claude
- Toate acestea sunt modele care procesează limbajul natural, adică preiau limbaj natural ca intrare și oferă limbaj natural ca ieșire
- Ceea ce vrem să vedem acum la BIA este cum s-a ajuns la aceste modele, mai exact baza de la care totul a urmat să se construiască
- Primul lucru interesant pe care vrem să îl vedem este cum se pregătește textul pentru antrenarea unui astfel de model care să genereze text
- Se mai pune problema dacă cu ce știm până acum putem rezolva problema procesării limbajului natural, sau dacă avem nevoie de lucruri mai sofisticate

Procesarea limbajului natural

- Mai întâi, trebuie să adunăm text
- Acest text poate fi adunat de oriunde, spre exemplu, mail-uri
- Începem să strângem text, ca acesta:

```
> Anyone knows how much it costs to host a web portal ?
```

```
>
```

```
Well, it depends on how many visitors youre expecting. This can be  
anywhere from less than 10 bucks a month to a couple of $100. You  
should checkout http://www.rackspace.com/ or perhaps Amazon EC2 if  
youre running something big..
```

```
To unsubscribe yourself from this mailing list, send an email to:  
groupname-unsubscribe@egroups.com
```


Procesarea limbajului natural

> Anyone knows how much it costs to host a web portal ?

>

Well, it depends on how many visitors youre expecting. This can be anywhere from less than 10 bucks a month to a couple of \$100. You should checkout <http://www.rackspace.com/> or perhaps Amazon EC2 if youre running something big..

To unsubscribe yourself from this mailing list, send an email to:
groupname-unsubscribe@egroups.com

- Facem niște preprocesări (facem toate literele mici, înlocuim link-urile cu un cuvânt unic, înlocuim adresa de mail cu un cuvânt unic, eliminăm punctuația)

anyone knows how much it costs to host a web portal

**well it depends on how many visitors youre expecting this can be anywhere from less than
bucks a month to a couple of you should checkout link or perhaps link if youre running
something big**

to unsubscribe yourself from this mailing list send an email to mail

Procesarea limbajului natural

anyone knows how much it costs to host a web portal

**well it depends on how many visitors youre expecting this can be anywhere from less than
bucks a month to a couple of you should checkout link or perhaps link if youre running
something big**

to unsubscribe yourself from this mailing list send an email to mail

- Atenție! Dacă eliminăm punctuația, vom realiza un model care nu va fi capabil să învețe regulile de punctuație
- Pentru simplitate, punctuația este ignorată, dar un model ca ChatGPT va ține cont de punctuație

Procesarea limbajului natural

anyone knows how much it costs to host a web portal

well it depends on how many visitors youre expecting this can be anywhere from less than bucks a month to a couple of you should checkout link or perhaps link if youre running something big

to unsubscribe yourself from this mailing list send an email to mail

- De asemenea, transformăm link-urile si adresele de e-mail într-un text unic gen „link” și „mail” pentru a încuraja modelul să nu facă overfitting
- Altfel, modelul ar fi fost forțat să facă overfitting, pentru că mereu ar fi dat de link-uri și mail-uri prea specifice, ce l-ar încurca
- În sens restrâns, de obicei pentru un model nu contează ce link există într-un text sau ce adresă de mail, pentru generalizare are nevoie să știe doar că ele există, nu și alte detalii
- Dar dacă vrem un model care înțelege link-uri și adrese de e-mail intrinsec, atunci avem nevoie să le lăsăm așa cum sunt

Procesarea limbajului natural

anyone knows how much it costs to host a web portal

**well it depends on how many visitors youre expecting this can be anywhere from less than
bucks a month to a couple of you should checkout link or perhaps link if youre running
something big**

to unsubscribe yourself from this mailing list send an email to mail

- Acum că avem textul preprocesat, avem nevoie de un mod prin care putem transforma textul în numere
- Motivul e că rețelele pe care le folosim funcționează pe bază de numere, nu de text
- Mai exact, termenul folosit este construirea unui vocabular

Procesarea limbajului natural

anyone knows how much it costs to host a web portal

**well it depends on how many visitors youre expecting this can be anywhere from less than
bucks a month to a couple of you should checkout link or perhaps link if youre running
something big**

to unsubscribe yourself from this mailing list send an email to mail

- Mai întâi, având acest text, trebuie să extragem așa-numitele tokens (unitatea de procesare a unui model bazat pe limbaj natural)
- Spre exemplu, pentru majoritatea modelelor de genul, se realizează o tokenizare la nivel de cuvânt (aka învățăm modelul relații între cuvinte)
- Alte modele mai complexe pot împărți textul după litere, sau alte modele împart fiecare cuvânt în alte subcuvinte și acelea devin token-uri

Procesarea limbajului natural

```
tokens = [  
    "anyone", "knows", "how", "much", "it", "costs", "to", "host", "a", "web", "portal",  
    "well", "it", "depends", "on", "how", "many", "visitors", "youre", "expecting", "this",  
    "can", "be", "anywhere", "from", "less", "than", "bucks", "a", "month", "to", "a",  
    "couple", "of", "you", "should", "checkout", "link", "or", "perhaps", "link", "if",  
    "youre", "running", "something", "big", "to", "unsubscribe", "yourself", "from",  
    "this", "mailing", "list", "send", "an", "email", "to", "mail"  
]
```

Procesarea limbajului natural

- Continuăm să strângem text de oriunde știm
- După acest pas, avem nevoie de frecvența cuvintelor (din tot corpul de text adunat - adică efectiv toate textele), pentru a „elimina” cuvintele care apar rar
- De fapt noi nu le eliminăm, doar le vom marca cu alt cuvânt, de exemplu „unknown”
- Motivul este că acele cuvinte care apar cel mai rar, s-ar putea să ducă modelul spre overfitting prin simplul fapt că nu avem destule exemplare cu acele cuvinte ca modelul să învețe
- Mai mult, chestia asta ne ajută să nu stocăm așa de multe cuvinte în memorie, ceea ce este un plus când memoria e limitată

Procesarea limbajului natural

```
token_counts = {'to': 5, 'a': 5, 'hosting': 3, 'for': 3, 'how': 2, 'it': 2, 'web': 2, 'portal': 2, 'visitors': 2, 'youre': 2, 'this': 2, 'be': 2, 'from': 2, 'than': 2, 'of': 2, 'link': 2, 'or': 2, 'anyone': 1, 'knows': 1, 'much': 1, 'costs': 1, 'host': 1, 'well': 1, 'depends': 1, 'on': 1, 'many': 1, ...}
```

- Putem considera, de exemplu, că toate cuvintele de frecvență 1 sunt irelevante, și să le marcăm ca unknown
- Textul ar deveni:

<unk> <unk> how <unk> it <unk> to <unk> a web portal <unk> it <unk> <unk> how <unk> visitors
youre <unk> this <unk> be <unk> from <unk> than <unk> a <unk> to a <unk> of <unk>
<unk><unk> link or <unk> link <unk> youre <unk> <unk> <unk> to <unk> <unk> fromthis <unk>
<unk> <unk> <unk> email to <unk> hosting a web portal for a <unk> <unk> of visitors <unk>
<unk> <unk> than <unk> <unk> <unk> <unk> to <unk> <unk> <unk> for <unk> <unk> <unk>
hosting <unk> be <unk> for <unk> <unk> <unk> <unk> <unk> or <unk> hosting <unk> <unk>
<unk>

Procesarea limbajului natural

<unk> <unk> how <unk> it <unk> to <unk> a web portal <unk> it <unk> <unk> how <unk> visitors
youre <unk> this <unk> be <unk> from <unk> than <unk> a <unk> to a <unk> of <unk>
<unk><unk> link or <unk> link <unk> youre <unk> <unk> <unk> to <unk> <unk> fromthis <unk>
<unk> <unk> <unk> email to <unk>hosting a web portal for a <unk> <unk> of visitors<unk>
<unk> <unk> than <unk> <unk> <unk> <unk> to <unk><unk> <unk> for <unk> <unk> <unk>
hosting<unk> be <unk> for <unk> <unk> <unk> <unk> <unk> or<unk> hosting <unk> <unk>
<unk>

- Bine, ăsta a fost un exemplu foarte simplu și de aceea avem multe <unk> în textul rezultat (dacă am antrena cu ce e mai sus, nu am ajunge foarte departe)
- De aceea, trebuie să strângem foarte multe date
- <unk> va reprezenta un cuvânt/token special, care va semnifica un „cuvânt neobișnuit”

Procesarea limbajului natural

```
token_counts = {'to': 5, 'a': 5, 'hosting': 3, 'for': 3, 'how': 2, 'it': 2, 'web': 2, 'portal': 2, 'visitors': 2, 'youre': 2, 'this': 2, 'be': 2, 'from': 2, 'than': 2, 'of': 2, 'link': 2, 'or': 2, 'anyone': 1, 'knows': 1, 'much': 1, 'costs': 1, 'host': 1, 'well': 1, 'depends': 1, 'on': 1, 'many': 1, ...}
```

- Trebuie să asociem un ID unic la fiecare cuvânt pentru a le trimite către un model
- De obicei, cuvântul <unk> are printre ID-urile cele mai mici, de exemplu o
- După ce știm cuvintele ordonate după frecvență și știm care sunt și cuvintele unknown, putem să asociem un ID unic la restul cuvintelor
- Spre exemplu, clasic ar fi să asociem ID crescător pornind de la cel mai frecvent cuvânt din acest dicționar și până la cel mai puțin frecvent
- În acest caz, ‘to’ ar primi ID 1, ‘a’ ar primi ID 2, ‘hosting’ ar primi ID 3 și așa mai departe

Procesarea limbajului natural

- După conversie, textul de mai devreme devine:

0	0	5	0	6	0	1	0	2	7	8	0	6	0	0	5	0	9	1	0	0	1	1	0		
1	2	0	1	3	0	1	4	0	2	0	1	2	0	1	5	0	0	0	1	6	1	7	0	1	6
0	1	0	0	0	0	1	0	0	1	3	1	1	0	0	0	0	0	1	0	3	2	7	8	4	
2	0	0	1	5	9	0	0	0	1	4	0	0	0	0	1	0	0	0	4	0	0	0	0	0	
3	0	1	2	0	4	0	0	0	0	0	1	7	0	3	0	0	0	0	0	0	0	0	0	0	

- Efectiv e un șir de numere
- Ce e mai sus e doar un exemplu, dar un corpus real va conține milioane de cuvinte
- De obicei, se împarte ce e mai sus în sub-texte de dimensiune fixă pentru antrenare

Procesarea limbajului natural

```
class Vocab:
    """Vocabulary for text."""
    def __init__(self, tokens=None, min_freq=0, reserved_tokens=None):
        if tokens is None:
            tokens = []
        if reserved_tokens is None:
            reserved_tokens = []
        # Sort according to frequencies
        counter = count_corpus(tokens)
        self._token_freqs = sorted(counter.items(), key=lambda x: x[1],
                                   reverse=True)

        # The index for the unknown token is 0
        self.idx_to_token = ['<unk>'] + reserved_tokens
        self.token_to_idx = {token: idx
                             for idx, token in enumerate(self.idx_to_token)}
        for token, freq in self._token_freqs:
            if freq < min_freq:
                break
            if token not in self.token_to_idx:
                self.idx_to_token.append(token)
                self.token_to_idx[token] = len(self.idx_to_token) - 1

    def __len__(self):
        return len(self.idx_to_token)

    def __getitem__(self, tokens):
        if not isinstance(tokens, (list, tuple)):
            return self.token_to_idx.get(tokens, self.unk)
        return [self.__getitem__(token) for token in tokens]

    def to_tokens(self, indices):
        if not isinstance(indices, (list, tuple)):
            return self.idx_to_token[indices]
        return [self.idx_to_token[index] for index in indices]

    @property
    def unk(self): # Index for the unknown token
        return 0

    @property
    def token_freqs(self): # Token frequencies
        return self._token_freqs

def count_corpus(tokens):
    """Count token frequencies."""
    # Here `tokens` is a 1D list or 2D list
    if len(tokens) == 0 or isinstance(tokens[0], list):
        # Flatten a list of token lists into a list of tokens
        tokens = [token for line in tokens for token in line]
    return collections.Counter(tokens)
```

Procesarea limbajului natural

- De obicei, se împarte ce e mai sus în sub-texte de dimensiune fixă pentru antrenare
- Spre exemplu, dacă dimensiunea fixă este 5 (adesea numită num_steps):

**[0 0 5 0 6], [0 1 0 2 7], [8 0 6 0 0], [5 0 9
10 0], [11 0 12 0 13], [0 14 0 2 0], [12 0 15
0 0], [0 16 17 0 16], [0 10 0 0 0], [1 0 0 13
11 0], [0 0 0 0 1 0], [3 2 7 8 4], [2 0 0 15 9],
[0 0 0 14 0], [0 0 0 1 0], [0 0 4 0 0], [0 3
0 12 0], [4 0 0 0 0], [0 17 0 3 0], [0 0]**

- O problemă apare la ultima secvență, care nu poate să aibă dimensiunea fixă 5
- În acest caz, fie o eliminăm, fie o păstrăm dar îi adăugăm un caracter special la sfârșit până o facem de dimensiunea cerută
- Acest caracter special poartă numele de padding

Procesarea limbajului natural

- O problemă apare la ultima secvență, care nu poate să aibă dimensiunea fixă 5
- În acest caz, fie o eliminăm, fie o păstrăm dar îi adăugăm un caracter special la sfârșit până o facem de dimensiunea cerută
- Acest caracter special poartă numele de padding
- În acest caz secvența de pe slide-ul precedent:

[0 0]

- Se transformă în:

[0 0 <id_pad> <id_pad> <id_pad>]

- Unde <id_pad> este un ID, ales de noi, care să semnifice acest caracter special

Procesarea limbajului natural

Se mai adaugă:

[0 0 5 0 6], [0 1 0 2 7], [8 0 6 0 0], [5 0 9
10 0], [11 0 12 0 13], [0 14 0 2 0], [12 0 15
0 0], [0 16 17 0 16], [0 10 0 0 0], [10 0 13
11 0], [0 0 0 0 1 0], [3 2 7 8 4], [2 0 0 15 9],
[0 0 0 14 0], [0 0 0 1 0], [0 0 4 0 0], [0 3
0 12 0], [4 0 0 0 0], [0 17 0 3 0], [0 0
<id_pad> <id_pad> <id_pad>]

<BOS> la început

<EOS> la final
la o secvență dacă se
dorește ca modelul să
învețe când să se
oprească singur din
generat text

- Totuși, nici acum datele noastre nu sunt pregătite în totalitate pentru antrenare
- Dacă am folosi datele numerice așa cum sunt ele acum, numeric, lucrul acesta adaugă un fel de ordine artificială în date ce în mod normal nu există pentru noi, oamenii
- Spre exemplu, dacă „câine” are ID 1, „șobolan” are ID 2 și „os” are ID 3, modelul ar putea învăța că „câine” este contextual mai apropiat de „șobolan” și nu de „os” doar pentru că 1 e mai aproape de 2 decât de 3

Procesarea limbajului natural

- Codificarea One-Hot rezolvă această problemă
- Pentru secvența [0, 4, 2, 9, 1] de exemplu, și un număr total de 10 cuvinte în vocabular, codificarea One-Hot realizează următoarea asociere:

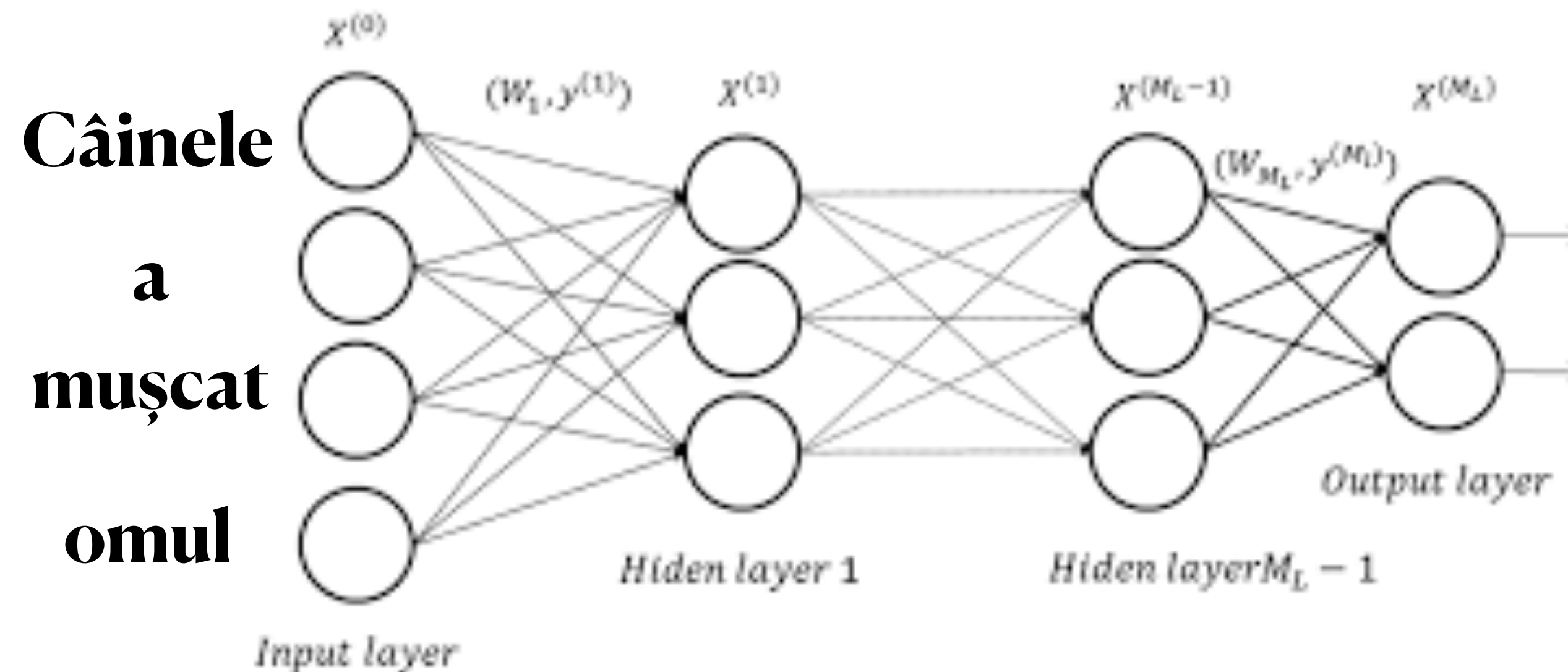


1	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0

- Practic, o linie din matrice reprezintă un cuvânt, iar coloanele reprezintă ce cuvânt este. Se pune un 1 pe poziția corespunzătoare din coloana cuvântului specific reprezentat de index-ul original

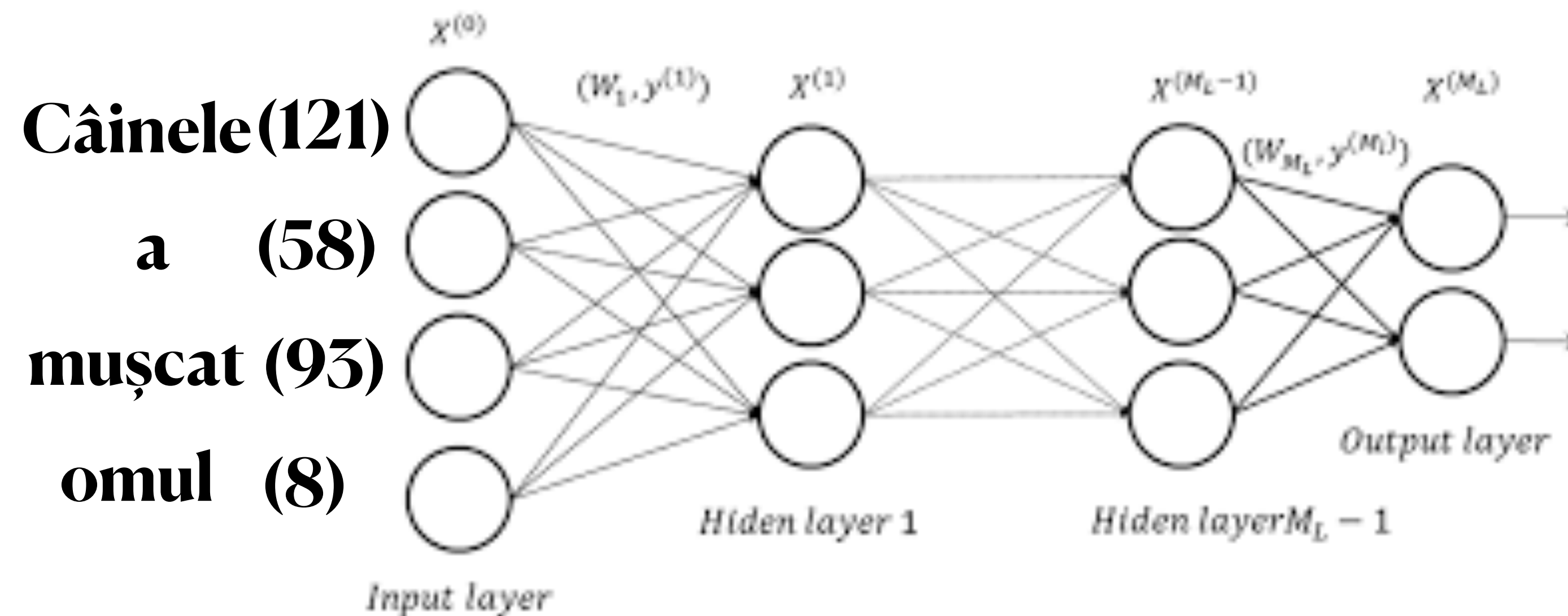
Procesarea limbajului natural

- Oare este un perceptron multistrat de ajuns să învățăm și să prezicem text?



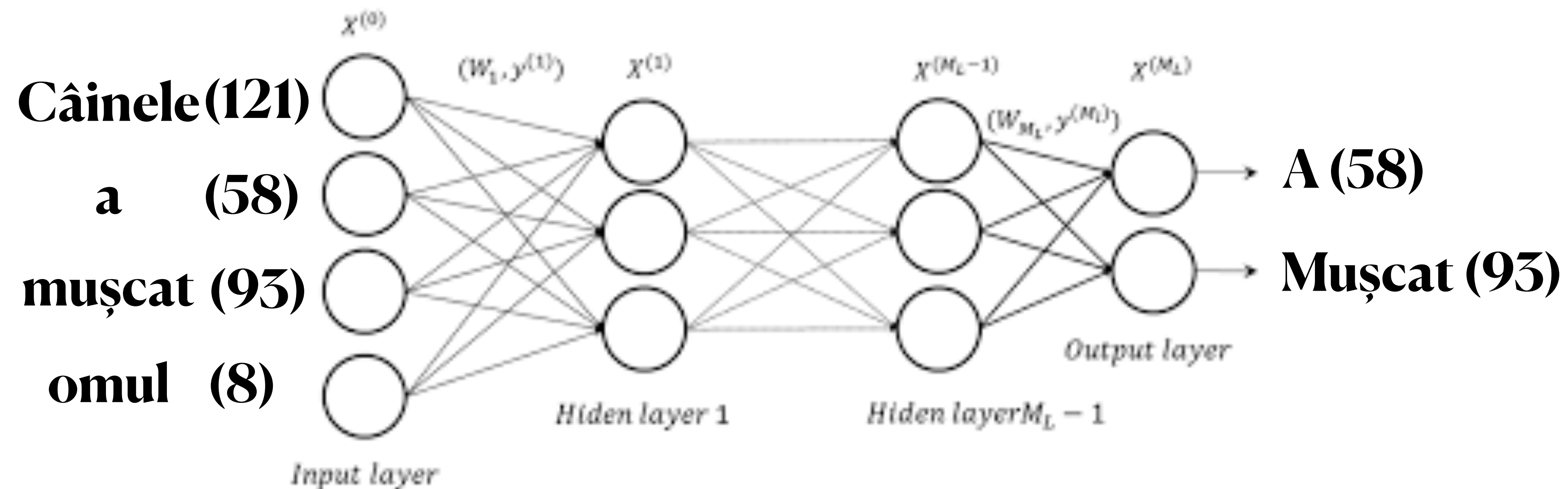
Procesarea limbajului natural

- Spre exemplu, de ce un perceptron multistrat nu e de ajuns?



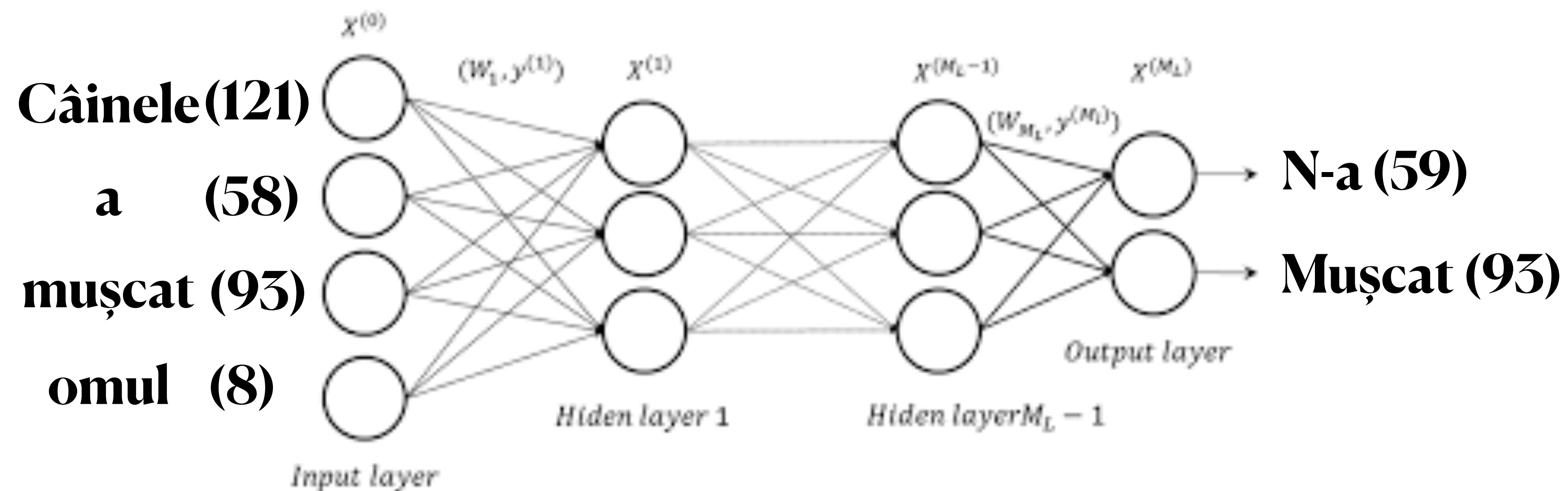
Procesarea limbajului natural

- Spre exemplu, de ce un perceptron multistrat nu e de ajuns?

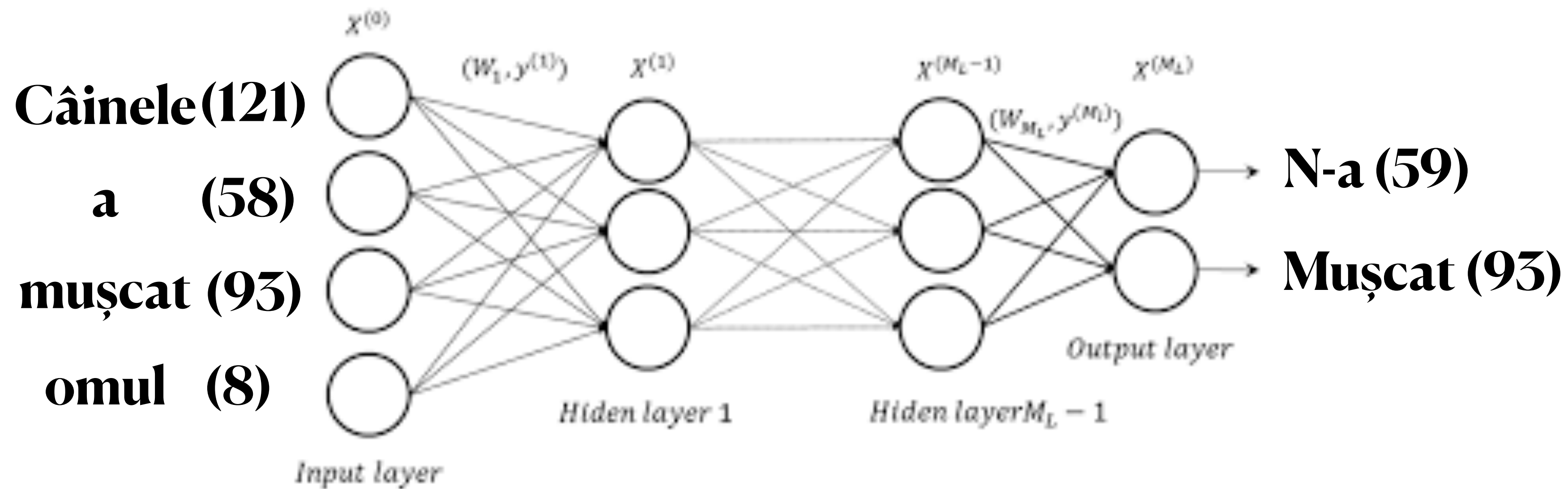


Procesarea limbajului natural

- Spre exemplu, de ce un perceptron multistrat nu e de ajuns?

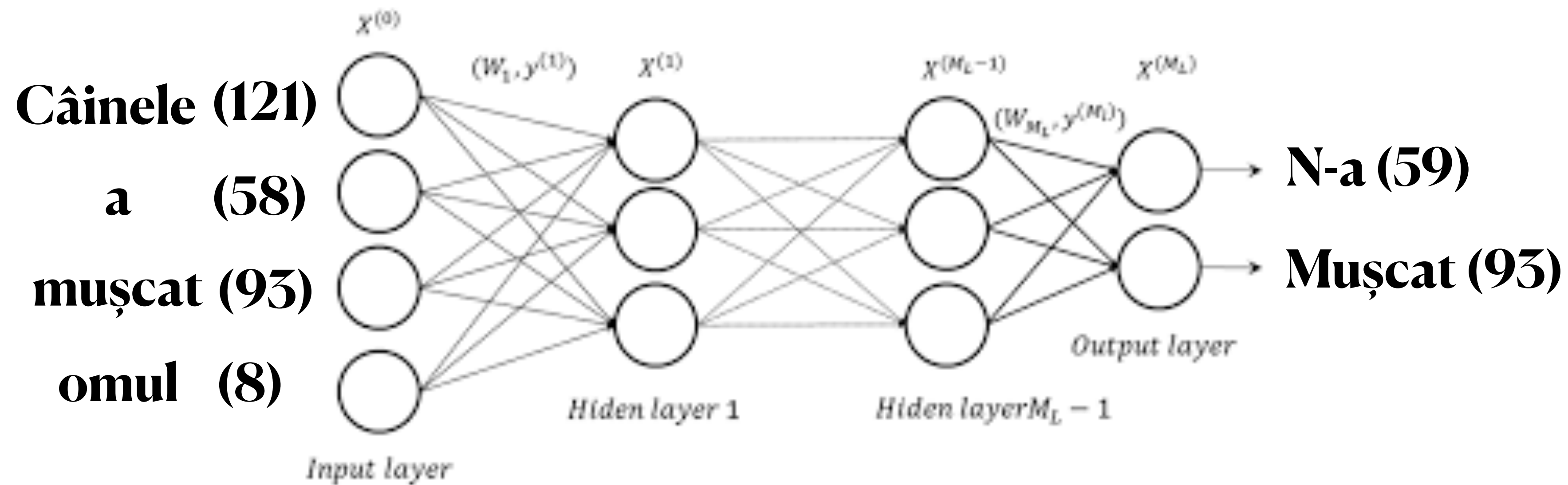


Procesarea limbajului natural



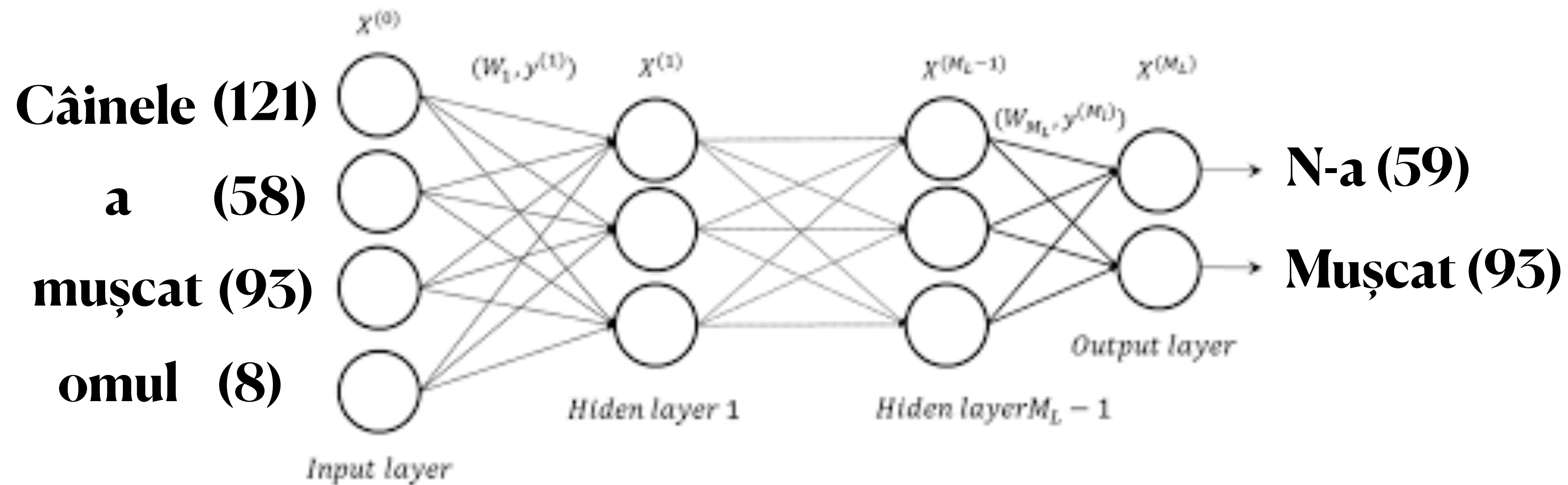
- Rețeaua de mai sus poate primi 4 intrări (adică 4 cuvinte), mai exact, fix 4 cuvinte, și produce ca ieșire 2 elemente, mai exact, fix 2 elemente
- Un singur dezavantaj se observă din start: rețeaua primește un număr fix de cuvinte ca intrare și poate produce un număr fix de cuvinte ca ieșire
- Nu există flexibilitate: pentru un MLP cu n intrări și m ieșiri, trebuie mereu să dăm n intrări ca intrare și mereu vom obține fix m ieșiri

Procesarea limbajului natural



- O a doua problemă este legată de felul în care rețeaua procesează cuvintele
- În acest caz, cuvintele de intrare sunt procesate independent, indiferent de ordinea lor în propoziția inițială
- Pe scurt, nu există relații între cuvinte. Modelul nu va putea învăța diferența dintre „câinele a mușcat omul” sau „omul a mușcat câinele”

Procesarea limbajului natural



- Avem cele două mari probleme:
 - Intrări fixe, ieșiri fixe
 - Neînțelegerea contextului în propoziții

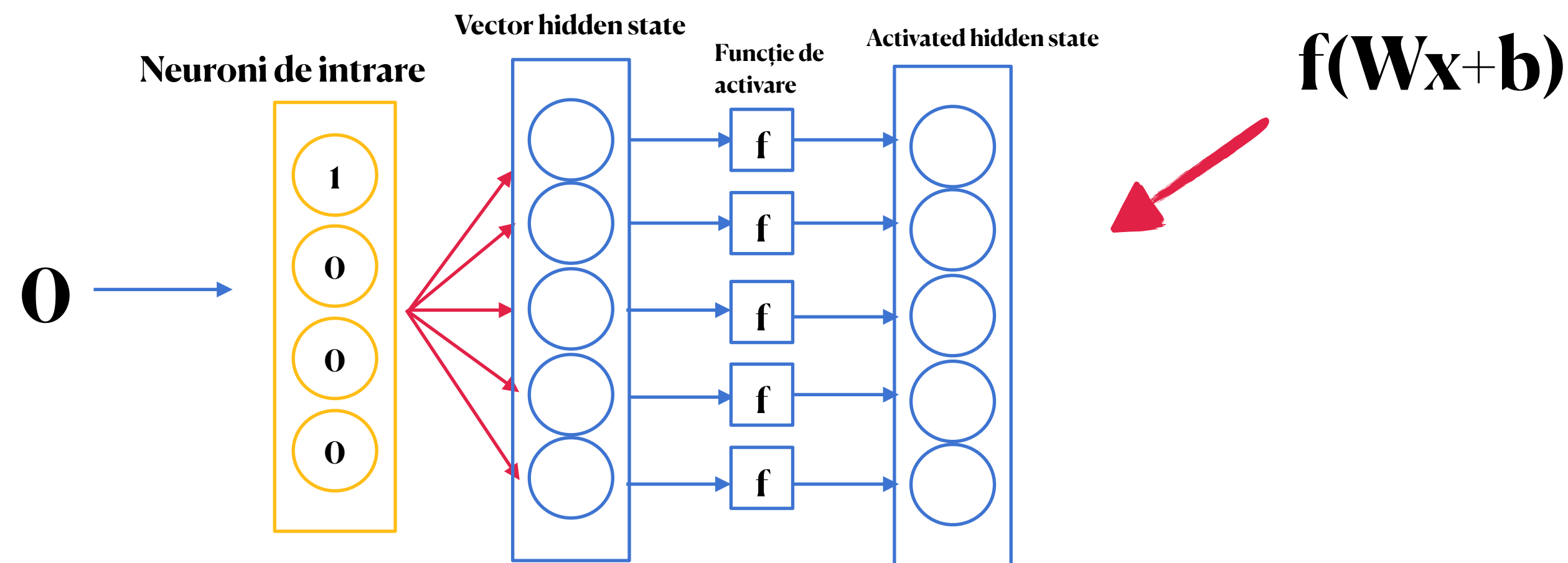
O parte din aceste probleme sunt rezolvate de rețelele neuronale recurente

Rețelele neuronale recurente

- Rețelele neuronale recurente au fost create pentru a rezolva problema modelării de relații secvențiale între caracteristicile primite ca intrare
- Mai exact, rezolvă problema procesării limbajului natural discutată mai devreme, dar nu numai aceasta, ci orice problemă care implică relații contextuale între caracteristici
- Spre exemplu, altă problemă care poate fi rezolvată folosind aceste tipuri de rețele este cea în care avem detalii despre pozițiile unei mingi în timp și vrem să prezicem poziția în viitor
- Secretul din spatele acestui tip de rețea este că au implementat conceptul de memorie secvențială, care există și în creierul uman
- Puteți să încercați să spuneți aflabetul în minte. După ce ați terminat, încercați să spuneți, tot în minte, alfabetul dar invers. E mai greu, nu? Asta pentru că ai memorat deja alfabetul normal ca pe o secvență

Retelele neuronale recurente

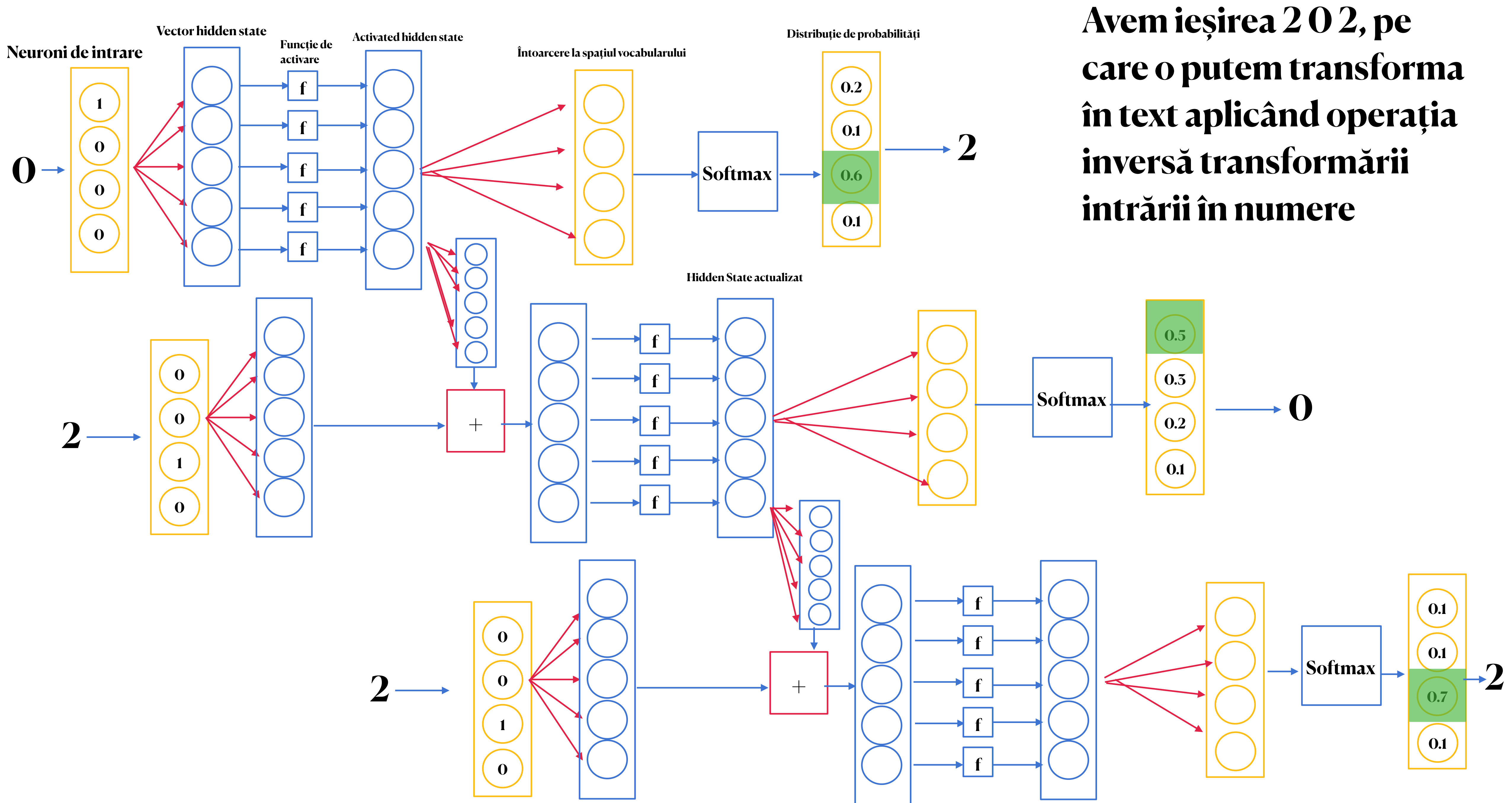
- Pentru simplitate, să zicem că avem un vocabular cu doar 4 cuvinte, haideți să vedem cum arată o rețea neuronală recurentă care primește ca intrare 3 cuvinte (0 2 2 - cuvinte deja codificate numeric)
- Vom observa că aceste rețele seamănă într-un fel cu MLP-urile



Cu galben va fi desenat spațiul vocabularului

Cu albastru va fi desenat spațiul „ascuns”

Cu roșu voi desena conexiuni fully-connected (adică toți neuronii dintr-un strat sunt conectați toți din stratul următor)



Retelele neuronale recurente

- Un astfel de model va fi antrenat încercând să minimizeze funcția de cost cross-entropy
- Adică va fi antrenat cu mai multe combinații prompt - ieșire și funcția de cost cross-entropy va face ca modelul să învețe să dea o probabilitate mai mare la acele cuvinte care sunt „most likely”
- Totuși, ca metrică de evaluare a performanței modelului se folosește metrica perplexity destul de des
- Perplexity este definit în felul următor:

$$\text{Perplexity} = e^{-\frac{1}{N} \sum_{i=1}^N \log P(c_i)}$$

- Unde N e numărul de cuvinte generate și $P(c_i)$ reprezintă probabilitatea celui de-al i-lea cuvânt generat iar log este logaritmul natural

Retelele neuronale recurente

$$\text{Perplexity} = e^{-\frac{1}{N} \sum_{i=1}^N \log P(c_i)}$$

- Unde N e numărul de cuvinte generate și $P(c_i)$ reprezintă probabilitatea celui de-al i -lea cuvânt generat iar $\log$ este logaritmul natural
- Spre exemplu, să zicem că modelul generează textul „Îmi place BIA”, cu $P(\text{„Îmi”}) = 0.5$, $P(\text{„place”}) = 0.2$ și $P(\text{„BIA”}) = 0.1$

$$\sum_{i=1}^N \log P(c_i) = \log(0.5) + \log(0.2) + \log(0.1) = -4.60$$

$$\text{Average} = -\frac{1}{N} \sum_{i=1}^N \log P(c_i) = -\frac{1}{3} \times (-4.60) \approx 1.535$$

$$\text{Perplexity} = e^{1.535} \approx 4.64$$

Retelele neuronale recurente

$$\sum_{i=1}^N \log P(c_i) = \log(0.5) + \log(0.2) + \log(0.1) = -4.60$$

$$\text{Average} = -\frac{1}{N} \sum_{i=1}^N \log P(c_i) = -\frac{1}{3} \times (-4.60) \approx 1.535$$

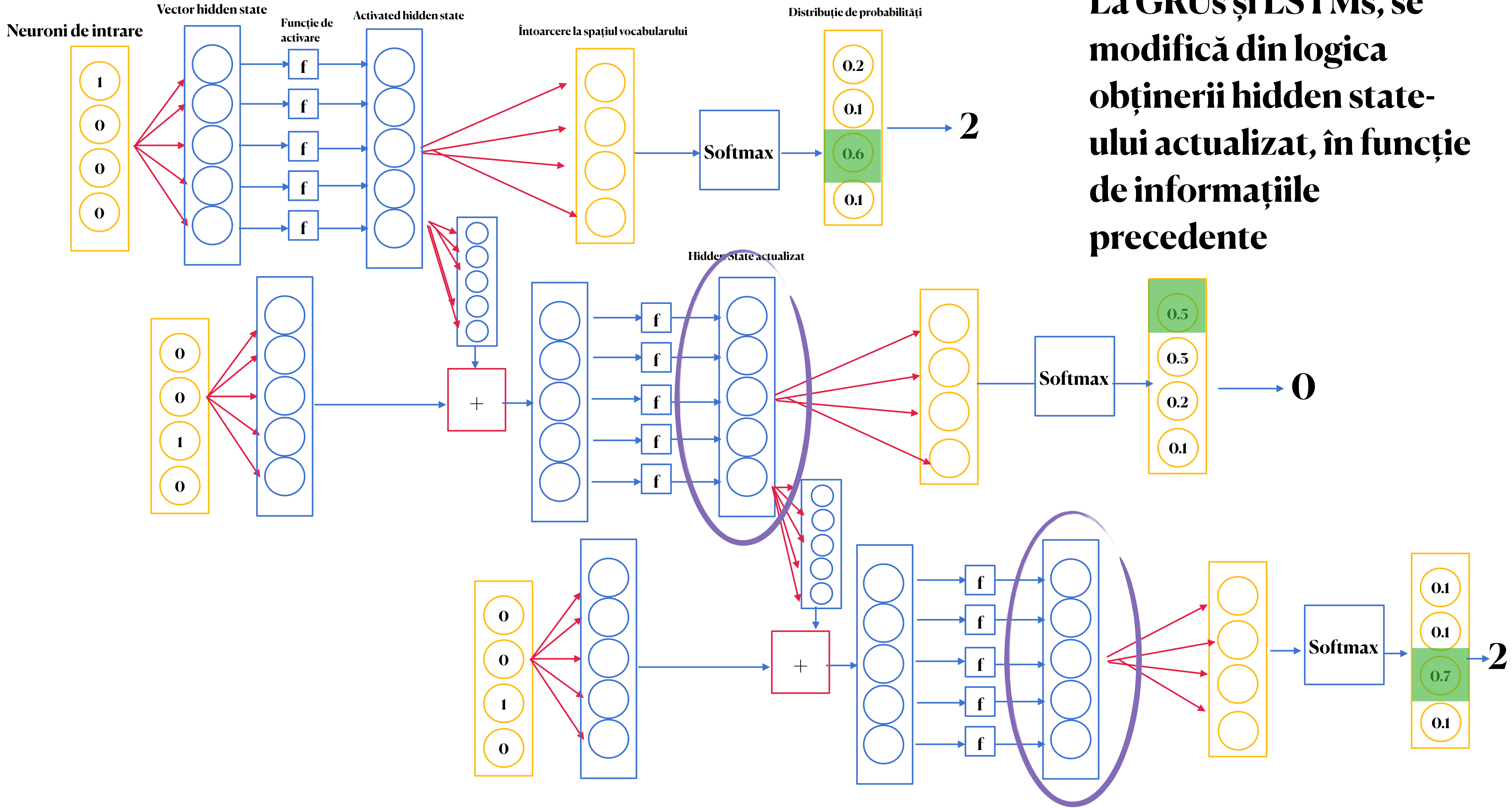
$$\text{Perplexity} = e^{1.535} \approx 4.64$$

- Rezultatul este 4.64, și este interpretat în felul următor: modelul era atât de nesigur încât atunci când încerca să prezică următorul cuvânt, avea de ales între aproximativ 4.64 alte cuvinte
- Interpretarea asta ne spune și scopul metricii: trebuie să tindă spre 0, pentru că cu cât tinde spre 0, cu atât modelul este mai sigur de prezicerea cuvântului următor

Rețelele neuronale recurente

- Totuși, există încă probleme cu rețelele neuronale recurente
- Cea mai mare problemă care s-a observat de cercetători este că memoria este de durată scurtă
- Mai exact, cu cât îi dăm modelului mai multe cuvinte, cu atât uită mai mult context de la începutul prompt-ului
- Spre exemplu, i-am putea zice „Who is Albert Einstein? Also, tell me about Thanos”
- Modelul s-ar putea să uite complet prima întrebare și să se focalizeze doar pe a doua cerință
- În general, pentru sarcinile care au nevoie de context mai larg, rețele neuronale recurente nu sunt folosite
- Sunt folosite mai mult surorile acestui tip de rețea, anume GRUs (Gated Recurrent Units) și LSTM (Long Short-Term Memory), care modifică arhitectura de pe slide-ul trecut ca să încerce să rezolve mai bine această problemă

La GRUs și LSTMs, se modifică din logica obținerii hidden state-ului actualizat, în funcție de informațiile precedente



Rețelele neuronale recurente

Atenție! Poate primi parametrul num_layers

```
num_hiddens = 256
rnn_layer = nn.RNN(len(vocab), num_hiddens)

class RNNModel(nn.Module):
    """The RNN model."""
    def __init__(self, rnn_layer, vocab_size, **kwargs):
        super(RNNModel, self).__init__(**kwargs)
        self.rnn = rnn_layer
        self.vocab_size = vocab_size
        self.num_hiddens = self.rnn.hidden_size
        self.linear = nn.Linear(self.num_hiddens, self.vocab_size)

    def forward(self, inputs, state):
        X = F.one_hot(inputs.T.long(), self.vocab_size)
        X = X.to(torch.float32)
        Y, state = self.rnn(X, state)
        output = self.linear(Y.reshape((-1, Y.shape[-1])))
        return output, state
    ...

net = RNNModel(rnn_layer, vocab_size=len(vocab))
```

- Atât de simplu se creează un RNN
- Clasa asta încorporează mai mulți pași, cum ar fi codificarea one-hot a intrării, transmiterea secvențială către RNN și după transpunerea rezultatului în spațiul vocabularului
 - Codificarea one-hot a intrării
 - Trimiterea intrării către straturile RNN
 - Y va avea dimensiunea hidden_space și îl transpunem în dimensiunea vocabularului
- output va conține toate cuvintele generate dar înainte de softmax

Rețelele neuronale recurente

```
num_hiddens = 256
rnn_layer = nn.RNN(len(vocab), num_hiddens)

class RNNModel(nn.Module):
    """The RNN model."""
    def __init__(self, rnn_layer, vocab_size, **kwargs):
        super(RNNModel, self).__init__(**kwargs)
        self.rnn = rnn_layer
        self.vocab_size = vocab_size
        self.num_hiddens = self.rnn.hidden_size
        self.linear = nn.Linear(self.num_hiddens, self.vocab_size)

    def forward(self, inputs, state):
        X = F.one_hot(inputs.T.long(), self.vocab_size)
        X = X.to(torch.float32)
        Y, state = self.rnn(X, state)
        output = self.linear(Y.reshape((-1, Y.shape[-1])))
        return output, state
    ...

net = RNNModel(rnn_layer, vocab_size=len(vocab))
```

rnn_layer poate fi înlocuit cu:

```
nn.GRU(num_inputs, num_hiddens)
```

- Dacă vrem să folosim GRUs

```
nn.LSTM(num_inputs, num_hiddens)
```

- Dacă vrem să folosim LSTMs
- La fiecare dintre acestea putem adăuga mai multe straturi

```
num_layers = 2
lstm_layer = nn.LSTM(num_inputs, num_hiddens, num_layers)
model = RNNModel(lstm_layer, len(vocab))
```

- Fiecare strat rafinează ce a învățat stratul precedent, practic la fel ca straturile ascunse din MLP

Retelele neuronale recurente

- Cu rețele neuronale recurente / GRUs / LSTMs rezolvăm:
 - Problema întârilor fixe
 - Problema înțelegerii contextului între cuvinte (at least, a part of it)
- Totuși, ieșirea nu este intuitivă: dacă îi dam 10 cuvinte ca intrare, vom primi 10 cuvinte ca ieșire (în general, N cuvinte de intrare produc N cuvinte de ieșire)
- Chiar dacă învățăm modelul să prezică un caracter special de genul EOS (End of Sequence), nu ne-ar ajuta, deoarece rezolvăm doar problema când ieșirea trebuie să fie mai mică în lungime decât intrarea, dar dacă lungimea ieșirii trebuie să fie mai mare decât intrarea?
- De asemenea, chiar și cu LSTMs, avem pierderi de memorie la secvențe lungi din cauza procesării secvențiale (LSTM-urile sunt cele mai performante tipuri de RNN)
- Săptămâna viitoare avem ultimul laborator de BIA, unde vom vedea cum arhitectura Transformer rezolvă toate problemele de până acum

That's it for today!

That's it for today!

Laboratory 10 – Exercises

1. Create an RNN with a single recurrent layer, an input size of 2 and a hidden size of 4. The input and the initial hidden state are tensors filled with random numbers of shape `(seq_length, batch, input_size)=(5,1,2)` and `(num_layers · num_directions, batch, hidden_size)=(1,1,4)`, respectively. Print the output of the RNN.
2. Use the text of H. G. Wells' "The Time Machine" to generate 20 characters, starting from the text "traveller". Use an RNN with two recurrent layers, each having 32 hidden units. Train the model for 200 epochs, with a learning rate of 1.5, a batch size of 30, and a number of steps of 10. Print the perplexity of the model at the last epoch.
3. Use the text of H. G. Wells' "The Time Machine" to generate 60 characters, starting from the text "time". Use an LSTM with two recurrent layers, each having 256 hidden units. Train the model for 400 epochs, with a learning rate of 2.5, a batch size of 25, and a number of steps of 10. Print the perplexity of the model at the last epoch.